Grounded Support Agent

A customer support agent for a UK wealth management firm, built to answer only what it can evidence — and to prove, in code, that it does.

The interesting part is not that an LLM can call tools. It is that a support assistant in a regulated industry must never invent a phone number, never paraphrase a case status into something rosier, and never leave a customer looping when it cannot help. This project treats those as engineering requirements with tests attached, rather than as instructions in a prompt and a hope.

```
python step10.py       # run the agent
python evaluate.py     # 52 tests, ~1 second, no API calls
```

> **Disclaimer.** A personal learning project, not affiliated with or endorsed by Evelyn Partners. The knowledge base is paraphrased from publicly available web pages; client records are invented. No real client data is used anywhere.

Contents

- Why this project
- Architecture
- Failure modes found in testing
- Testing strategy
- Design decisions and trade-offs
- Known limitations
- Running it

Why this project

Retrieval-augmented chatbots are easy to demo and hard to trust. A demo answers a question fluently; a trustworthy system knows what it does not know, refuses to embellish, and fails loudly rather than plausibly.

Financial services makes those distinctions concrete. Quoting a case status inaccurately is a customer-relations problem. Answering "should I move my pension into equities?" is a regulatory one. So the design question driving this build was:

> Which guarantees can live in a prompt, and which must live in code?

The answer, repeatedly, was **code**. Every hard guarantee here is enforced in Python, because a system prompt is a strong suggestion and nothing more — something demonstrated below by a model that broke a rule marked MUST, and was right to.

Architecture

Three tools, and a loop that runs while the model asks for them.

Tool	Purpose
`search_knowledge_base`	12 public FAQ entries — offices, complaints, services, portal
`look_up_client_query`	Case status by reference (synthetic data)
`escalate_to_adviser`	Hand-off to a human, mandatory for advice topics

Request lifecycle

```
user question
     │
     ├─ expire volatile results from previous turns      (staleness)
     │
     ├─ tool_choice="any" on step 0                      (no answering from memory)
     │
     ├─ RETRIEVAL
     │    ├─ LLM query expansion                          (meaning survives
rewording)
     │    ├─ BM25 scoring: IDF + length normalisation     (rare words decide)
     │    ├─ relevance floor                              (silence beats noise)
     │    └─ LLM rerank of top-5 candidates               (reading beats counting)
     │
     ├─ generation, constrained by STATUS_VERBATIM fields
     │
     ├─ CHECK 1: citation guard — deterministic           (URLs and numbers)
     ├─ CHECK 2: groundedness — independent LLM           (semantic drift)
     ├─ CHECK 3: escalation policy — deterministic        (clarify once, then
escalate)
     │
     └─ answer + audit trail of raw evidence
```

Checks run cheapest-and-most-certain first. The deterministic citation guard runs before the LLM fact-checker, because a `re.findall` costs nothing and cannot be argued with.

Two-stage retrieval

Lexical scoring is fast and cheap but counts words. Reranking is slower and costs a call but reads meaning. Combining them — retrieve widely, filter by comprehension — is the

standard pattern in production search, and it removed the need for an embedding model and its dependency weight entirely.

Failure modes found in testing

Each of these was discovered by running the system, not by reading the code. They are listed because the debugging is the substance of the project.

1. Paraphrase drift

Symptom. Source data said `Summary letter posted to your address.` The bot wrote *"a summary letter covering the outcome has been posted to your address on file."*

Why it matters. Harmless here. Not harmless when `Response due by 6 Aug` becomes *"you'll hear by the 6th"* — a target silently upgraded to a promise.

Why guardrails missed it. The tool was called correctly and the data was correct. Drift happened afterwards, during writing.

Fix. Three layers: a named `STATUS_VERBATIM` field for the prompt to point at (far stronger than "be faithful"), an explicit rule against converting targets into guarantees, and an independent fact-checker that reads the finished answer back against its evidence. The checker is a separate call because a model asked to critique its own output defends it.

2. Retrieval that manufactured its own relevance

Symptom. Asked about a Manchester office, query expansion returned `bristol` — a city never mentioned by the customer.

Cause. The expansion prompt was given the knowledge base topic list "for context". It suggested a term from the corpus, retrieval then matched its own suggestion, and the resulting confidence was circular.

Fix. The expander no longer sees the corpus it is about to search, and is explicitly barred from inventing place names, brands or numbers. The match score for that query fell from 0.76 to 0.43 — lower, and honest.

3. Extrapolated URLs

Symptom. Given `https://www.evelyn.com/offices/bristol/`, the bot told the customer to visit `evelyn.com/offices` — a parent path it inferred.

Why the LLM fact-checker passed it. It looked right, and it is a substring of a URL that was supplied.

Fix. `check_citations()` — deterministic. Every URL and number in the answer must match the evidence exactly after normalisation. Truncating a path is caught; so is an invented phone number. Code cannot be persuaded that something looks plausible.

4. A safety check that failed open

Symptom. The fact-checker crashed on `Extra data: line 2 column 1`, then logged `[verified: grounded]`.

Why it matters. A check that fails silently while reporting success is worse than no check, because it is trusted.

Cause. `json.loads()` on model output that contained valid JSON followed by a friendly sentence.

Fix. A brace-matching extractor that tolerates code fences, trailing prose and nested braces in strings, plus three honest states — `grounded`, `unsupported`, `unavailable` — with the unavailable case surfaced to the reader.

5. A MUST rule the model correctly ignored

Symptom. The tool description states that any `NO_MATCH` **must** escalate. Given the single word `quiz`, the model asked a clarifying question instead.

The interesting part. That was better than the rule. Escalating one ambiguous word puts a nonsense ticket in front of a paid adviser.

Fix. Not to enforce the rule harder, but to correct it, then enforce the corrected version in code: **clarify once, then escalate.** A second unresolved turn escalates from Python regardless of what the model decides, so a customer can never be left circling.

When a model overrides your rule and is right, the rule was wrong.

6. A patched answer that contradicted itself

Symptom. The policy appended *"I have passed this to an adviser"* to an answer that had just offered *"or I can pass this to an adviser?"*

Cause. The model wrote its reply believing escalation had not happened; the appended sentence contradicted that assumption.

Fix. Regenerate rather than concatenate. Escalation now triggers a rewrite with the new fact supplied, producing one coherent message.

Generated prose cannot be patched by string concatenation — if the premise changes, the text must be rewritten.

7. A test suite that passed by luck

Symptom. Raising `RELEVANCE_FLOOR` from 0.15 to 0.9 — a six-fold change that should devastate retrieval — failed only one test in 51.

Cause. The surviving tests used queries scoring 1.02 and 0.91. They cleared the raised floor coincidentally. One passed by 0.01.

Fix. A test that asserts the *margin* between real match scores and the floor, not merely the outcome. The same broken configuration now fails with a diagnostic that names the cause: `best 0.91 vs floor 0.9`.

A passing test proves that input works, not that the system works.

Testing strategy

`evaluate.py` **— 52 tests, about one second, no API calls.**

The core technique is a **scripted fake model**. Behaviour is non-deterministic: asking `quiz` twice produced two different execution paths, so the policy-escalation branch went untested purely by chance. A fake client that returns pre-written responses forces every branch on demand, including ones a well-behaved model would never take.

Layer	Covers	Cost
Unit tests	Ranking, citation guard, JSON extraction, expiry, policy state machine, rerank failure modes	free
Agent scenarios	Full `run_agent()` against the fake model: verbatim quoting, citation rewrites, forced escalation, greeting handling, crashed tools	free
Live tests (`--live`)	Real questions through the real API	opt-in

Failure modes are tested as first-class cases: helper model down, malformed JSON, out-of-range indexes, a tool raising an exception. The system degrades rather than breaking, and says so.

The suite also caught a bug while being written: a hardcoded fallback phone number in an error message is an unsupported claim like any other, and the citation guard stripped it.

Design decisions and trade-offs

Reranking instead of embeddings. A vector store and sentence-transformers would add roughly 2 GB of dependencies for a 12-entry corpus, and PyTorch wheels lag new Python releases. Query expansion plus LLM reranking achieves semantic matching using the API already in use. For a corpus in the thousands, embeddings would win on cost and latency.

Selective staleness. Case statuses expire from conversation history; FAQ content does not. Wiping static content would cost calls and latency for no accuracy gain. Volatility is a property of the tool, declared once in `VOLATILE_TOOLS`.

Forced tool use, with an exception. `tool_choice="any"` on the first step prevents answering from parametric memory. A hardcoded pleasantries list exempts greetings, because searching the knowledge base for "hello" is pure waste.

Everything is a flag. `USE_RERANK`, `VERIFY_ANSWERS`, `CHECK_CITATIONS`, `FORCE_TOOL_FIRST` toggle independently. A fully guarded answer can cost five API calls. That is right for a regulated firm and wrong for most products, so the trade-off is explicit rather than baked in.

Auditability over polish. Every answer carries its raw evidence. Terse and traceable beats fluent and unverifiable.

Known limitations

Stated plainly, because a system whose weaknesses are documented is safer than one whose weaknesses are unknown.

- **Non-determinism is irreducible.** Identical inputs can take different paths. This is why every hard guarantee lives in Python and why the audit trail exists.
- **Retrieval quality is bounded by the corpus.** Twelve entries is a demonstration, not coverage. Retrieval quality is downstream of content quality, always.
- **Negation is handled by the reranker, not the index.** Lexical scoring cannot see that *"not branch opening hours"* is a denial. Reranking catches it in testing; if the rerank call fails, the fallback is lexical order.
- **The fact-checker is itself a model.** It can be wrong in both directions. It is a second opinion, not a proof — which is why the deterministic guard runs first and independently.
- **Single-user, in-memory, no persistence.** No session store, no concurrency, no rate limiting. Not production infrastructure.

Running it

Requires Python 3.10+ and an Anthropic API key.

```bash
python -m venv .venv
.venv\Scripts\activate          # Windows;  source .venv/bin/activate on macOS/Linux
pip install anthropic python-dotenv
```

Create `.env` in the project root:

```
ANTHROPIC_API_KEY=sk-ant-...
```

Then:

```bash
python step10.py                # interactive agent
python evaluate.py              # test suite, free
python evaluate.py --live       # includes real API calls
```

`evaluate.py` exits non-zero on failure, so it drops straight into CI.

Files

File	Purpose
`step10.py`	The agent — tools, retrieval, guards, policy, loop
`knowledge_base.py`	12 sourced entries, each with a traceable URL
`evaluate.py`	52 tests, fake model, optional live suite

Try these

Input	Demonstrates
`What's the status of EP-1001?`	Verbatim quoting, no embellishment
`Should I move my pension into equities?`	Immediate escalation, zero advice
`Can I visit a branch on Sunday?`	Reranker rejecting a negation match
`Do you have a Manchester office?`	Honest uncertainty, no invented URL
`quiz` then `just the quiz thing`	Clarify once, then policy-enforced escalation

What I would do next

1. **Grow and version the knowledge base**, with freshness metadata and a scheduled re-verification job. The current bottleneck is content, not code.
2. **Swap in embeddings** once the corpus outgrows exhaustive lexical scoring, keeping the reranker as the second stage.
3. **Persist the audit trail** to structured logs, so escalation rates, `NO_MATCH` rates and rewrite frequency become monitorable metrics rather than terminal output.
4. **Expand the live suite** into a labelled regression set with pass thresholds, run before every change.